

SOSAround

Documentazione tecnica — Architettura, codice e logica

Applicazione client/server React Native (Expo) + Node

Generato il 18/06/2026 · Clean Architecture · Express + Socket.IO + SQLite

Indice

1. 1. Cos'è SOSAround
2. 2. Architettura generale (monorepo client/server)
3. 3. La Clean Architecture lato client
4. 4. Il dominio: entità e interfacce
5. 5. Il server (backend Node)
6. 6. Il client (app Expo/React Native)
7. 7. Comunicazione client ↔ server (REST + WebSocket)
8. 8. I flussi principali passo-passo
9. 9. Riferimento delle classi principali
10. 10. Glossario

1. Cos'è SOSAround

SOSAround è una piattaforma per la **gestione tempestiva degli smarrimenti** (animali, oggetti). Chi perde qualcosa registra un *Smart ID* del bene e apre una *segnalazione* geolocalizzata; gli altri utenti vicini ricevono una notifica e possono **contattare il proprietario** per coordinare il ritrovamento, in tempo reale.

Funzionalità principali

- **Smart ID**: registrazione di un bene (nome, tipo, foto, codice/microchip, QR).
- **Segnalazione**: allerta geolocalizzata con raggio di prossimità.
- **Bacheca community + Mappa**: elenco e mappa delle segnalazioni attive.
- **Chat 1:1 in tempo reale** tra proprietario e ogni potenziale ritrovatore.
- **Notifiche realtime** cross-device (nuove segnalazioni / nuovi messaggi).

2. Architettura generale (monorepo client/server)

Il progetto è un **monorepo** con due applicazioni indipendenti:

```
Progetto/
├── client/      App mobile Expo / React Native (Clean Architecture)
│   ├── App.tsx    bootstrap app, sessione, notifiche realtime
│   ├── index.js   entry point Expo (registra App)
│   └── src/
│       ├── domain/    regole di business pure (entità, interfacce)
│       ├── application/ use case (orchestrano il dominio)
│       ├── infrastructure/ implementazioni: API HTTP, socket, store Redux
│       └── presentation/ schermate, componenti, navigazione
└── server/      Backend Node: Express + Socket.IO + SQLite
    └── src/
        ├── index.ts    bootstrap server HTTP + WebSocket
        ├── db.ts        apertura SQLite + schema
        ├── auth/jwt.ts   firma/verifica token JWT + middleware
        ├── realtime/io.ts Socket.IO (stanze + emit eventi)
        └── routes/       endpoint REST (auth, users, smartid, ...)
```

Perché client/server. In origine l'app usava *Supabase* (BaaS) per database, autenticazione e realtime. È stato sostituito da un **server proprio** per avere il pieno controllo della logica e dei dati: REST per le operazioni CRUD, WebSocket (Socket.IO) per il tempo reale, SQLite per la persistenza, JWT per l'autenticazione.

Stack tecnologico

Strato	Tecnologia	Ruolo
Client UI	React Native + Expo	App mobile multiplatforma
Stato client	Redux Toolkit	Store globale (auth, segnalazioni, messaggi, smartId)
Trasporto	fetch (REST) + socket.io-client	Chiamate HTTP e canale realtime
Server	Express	API REST
Realtime	Socket.IO	Eventi push (segnalazioni, messaggi)
Database	SQLite (<code>node:sqlite</code>)	Persistenza, senza moduli nativi
Auth	JWT + bcrypt	Token firmati, password con hash

3. La Clean Architecture lato client

Il client segue la **Clean Architecture**: il codice è diviso in cerchi concentrici dove le dipendenze puntano sempre *verso l'interno*. Il dominio non conosce React né la rete; l'infrastruttura implementa le interfacce del dominio.

Livello	Cartella	Contenuto	Dipende da
Dominio	<code>domain/</code>	Entità + interfacce repository (contratti)	nulla
Applicazione	<code>application/</code>	Use case (logica applicativa)	dominio
Infrastruttura	<code>infrastructure/</code>	API HTTP, socket, Redux, servizi	dominio + applicazione
Presentazione	<code>presentation/</code>	Schermate, componenti, navigazione	infrastruttura

Vantaggio pratico. Passare da Supabase al server proprio ha richiesto di cambiare *solo* l'infrastruttura (i repository e il realtime): dominio, use case e gran parte della UI sono rimasti invariati, perché dipendono dalle *interfacce*, non dall'implementazione.

4. Il dominio: entità e interfacce

File: `client/src/domain/entities/index.ts`. Sono semplici interfacce TypeScript, senza logica di framework.

Entità	Descrizione	Campi chiave
Utente	Account registrato	idUtente, nome, email, punteggioReputazione, pushToken
SmartId	Bene registrato	idBene, idProprietario, tipo, nome, fotoUrl, stringQrCode, codiceIdentificativo
Segnalazione	Allerta di smarrimento	idSegnalazione, idBene, idSegnalatore, stato, latitudine, longitudine, raggioProssimita, nomeBene
Messaggio	Messaggio di chat	idMessaggio, idSegnalazione, idMittente, idDestinatario, testo, timestamp

Enum: **TipoBene** (ANIMALE/OGGETTO/ALTRO), **StatoSegnalazione** (ATTIVO/RITROVATO/SCADUTO).

Interfacce repository (contratti)

Definiscono *cosa* serve, non *come*. Le implementazioni concrete (**Api*Repository**) parlano col server.

```
ISmartIdRepository      findById / findByCodiceIdentificativo / findByProprietario /
                        findByQrCode / save / update / delete
ISegnalazioneRepository findById / findByBene / findAttive / findInRaggio /
                        save / updateStato / chiudi
IMessaggioRepository    findByConversazione / save
IUtenteRepository       findById / savePushToken
```

5. Il server (backend Node)

5.1 Bootstrap — `src/index.ts`

Crea l'app Express, monta i router, avvia il server HTTP e collega Socket.IO sullo stesso server.

```
const app = express();
app.use(cors());
app.use(express.json());
app.use('/auth', authRouter);
app.use('/users', usersRouter);
app.use('/smartid', smartIdRouter);
app.use('/segnalazioni', segnalazioniRouter);
app.use('/messaggi', messaggiRouter);
app.use('/conversazioni', conversazioniRouter);

const server = http.createServer(app);
initIo(server);           // Socket.IO sullo stesso server HTTP
server.listen(PORT);      // default :4000
```

5.2 Database — `src/db.ts` e `schema.sql`

Usa il modulo **integrato di Node** `node:sqlite` (niente compilazione nativa). All'avvio apre/crea il file `sosaround.db` ed esegue lo schema (idempotente con `IF NOT EXISTS`). Tabelle: `utente`, `smartid`, `segnalazione`, `messaggio`.

```
export const db = new DatabaseSync(dbPath);
db.exec('PRAGMA journal_mode = WAL;');
db.exec('PRAGMA foreign_keys = ON;');
db.exec(fs.readFileSync('schema.sql'));
```

5.3 Autenticazione — `src/auth/jwt.ts`

- `firmaToken({ idUtente })` → genera un JWT firmato (scadenza 30 giorni).
- `verificaToken(token)` → verifica e restituisce il payload, o `null`.
- `authGuard` → *middleware* Express: legge l'header `Authorization: Bearer <token>`, valida e popola `req.idUtente`; altrimenti risponde `401`.

Le password non sono mai salvate in chiaro: `bcrypt.hash` in registrazione, `bcrypt.compare` al login.

5.4 Realtime — `src/realtime/io.ts`

Inizializza Socket.IO con autenticazione JWT all'handshake. I client entrano in **stanze** (*room*) per ricevere solo gli eventi pertinenti. Il server emette eventi dopo le scritture REST.

```
// handshake autenticato col token JWT
io.use((socket, next) => {
  const payload = verificaToken(socket.handshake.auth?.token);
  if (!payload) return next(new Error('Non autorizzato'));
  socket.data.idUtente = payload.idUtente; next();
});

// stanze per la chat 1:1 (chiave = "idSegnalazione:idRitrovatore")
socket.on('join', (key) => socket.join('chat:' + key));

// emit dopo l'INSERT in REST:
emitNuovaSegnalazione(row)           → io.emit('segnalazione:nuova', row)
emitNuovoMessaggio(chiave, row)       →
io.to('chat:'+chiave).emit('messaggio:nuovo', row)
                                     io.emit('messaggio:nuovo:global', row)
```

Da Supabase a Socket.IO. Dove prima c'erano i `postgres_changes` di Supabase, ora è il *nostro* server a emettere esplicitamente gli eventi subito dopo aver salvato i dati.

5.5 Le route REST

Metodo & path	Cosa fa
POST /auth/register	Crea utente (hash password), ritorna { token, utente }
POST /auth/login	Verifica credenziali, ritorna { token, utente }
GET /auth/me	Utente corrente dal JWT (bootstrap sessione)
GET /users/:id	Profilo pubblico utente
PATCH /users/:id/push-token	Salva il token push Expo
GET/POST/PATCH/DELETE /smartid	CRUD beni + ricerca per proprietario/codice/QR
GET /segnalazioni	Attive, per bene, o in raggio (?lat&lng&raggio)
POST /segnalazioni	Crea + emette segnalazione:nuova
POST /segnalazioni/:id/chiudi	Stato → RITROVATO + emette segnalazione:aggiornata
GET /messaggi? segnalazione&ritrovatore	Messaggi della conversazione 1:1
POST /messaggi	Salva + emette nella stanza della conversazione
GET /conversazioni	Elenco conversazioni 1:1 dell'utente autenticato

La ricerca **geografica** non usa più PostGIS: il server filtra le segnalazioni attive con la formula *haversine* in JavaScript (`src/geo.ts`).

6. Il client (app Expo/React Native)

6.1 Bootstrap — `App.tsx`

- **Sessione:** all'avvio `AuthService.getSessioneAttiva()` ricarica il JWT dal disco e recupera l'utente; se valido → `LoginSuccess` + connessione socket.
- **Notifiche realtime:** il componente `NotificheRealtime` ascolta `segnalazione:nuova` e `messaggio:nuovo:global` e mostra una notifica locale (funziona anche in Expo Go, dove le push remote non sono disponibili).
- **Push token:** in development build registra il token Expo e lo salva sul server.

6.2 Store Redux — `infrastructure/store`

Slice	Stato
<code>authSlice</code>	utente loggato, loading, error
<code>segnalazioneSlice</code>	<code>attive[]</code> , loading, avviso
<code>messaggioSlice</code>	<code>conversazioni</code> : mappa "idSegnalazione:idRitrovatore" → <code>Messaggio[]</code>
<code>smartIdSlice</code>	beni dell'utente

Chiave delle conversazioni. La chat è indicizzata per coppia (`segnalazione + ritrovatore`) e non solo per segnalazione: così ogni ritrovatore ha una conversazione **1:1** separata col proprietario, e non una chat di gruppo.

6.3 Accesso ai dati — `infrastructure/api` e `repositories/api`

`apiClient.ts` è un piccolo wrapper su `fetch`: aggiunge l'header `Authorization` col JWT, serializza/deserializza JSON e **ricava l'URL del server** automaticamente dall'host con cui Expo serve l'app (così è sempre raggiungibile dal telefono).

I `Api*Repository` implementano le interfacce del dominio convertendo tra le righe del database (snake_case) e le entità (camelCase). Esempio (`ApiSegnalazioneRepository`):

```
async findInRaggio(lat, lng, raggioKm) {
  const rows = await api.get(`/segnalazioni?lat=${lat}&lng=${lng}&raggio=${raggioKm}`);
  return rows.map(rowToSegnalazione); // Row(snake_case) → Segnalazione(camelCase)
}
```

6.4 Realtime client — `realtime/socketClient.ts`

Espone un **singleton** Socket.IO connesso col JWT corrente: `getSocket()`, `connettiSocket()` (al login), `disconnettiSocket()` (al logout). Le schermate si iscrivono agli eventi e si discrivono allo smontaggio.

6.5 Schermate principali — presentation/screens

Schermata	Ruolo
AuthScreen	Login / registrazione
MappaScreen	Mappa delle segnalazioni attive (pin + chat)
BachecaScreen	Elenco segnalazioni; "Contatta" (ritrovatore) / "Ritrovato" (proprietario)
NuovaSegnalazioneScreen	Crea segnalazione con pin su mappa (anche a tutto schermo)
NuovoSmartIdScreen	Registra un bene (foto da fotocamera o galleria)
MessaggiListScreen	Elenco conversazioni 1:1 (da /conversazioni)
ChatScreen	Chat 1:1 in tempo reale
VaultScreen / DettaglioSmartId	Archivio beni dell'utente e dettaglio

7. Comunicazione client ↔ server

Due canali complementari:

- **REST** per leggere/scrivere dati (richiesta→risposta). Autenticato col JWT in header.
- **WebSocket** per il tempo reale (il server "spinge" gli eventi ai client). Autenticato col JWT all'handshake.

Esempio combinato: il client *salva* un messaggio via **POST /messaggi** (REST); il server, dopo l'INSERT, *emette* **messaggio:nuovo** nella stanza della conversazione (WebSocket), e l'altro partecipante lo riceve all'istante senza ricaricare.

8. I flussi principali passo-passo

8.1 Registrazione / Login

- 1 L'utente compila `AuthScreen`; `AuthService.validaEmail/validaPassword` controllano i campi.
- 2 `AuthService.login()` chiama `POST /auth/login`; il server verifica con bcrypt e ritorna `{ token, utente }`.
- 3 Il token viene salvato in **SecureStore** (storage cifrato) e tenuto in memoria per gli header.
- 4 Dispatch `loginSuccess(utente)` → la UI passa alle tab principali; `connettiSocket()` apre il canale realtime.

8.2 Creazione di una segnalazione

- 1 `NuovaSegnalazioneScreen`: l'utente sceglie il punto sulla mappa e descrive il bene.
- 2 `AllertaController.attivaAllerta()` esegue `AttivaAllertaUseCase`: crea lo Smart ID e salva la `Segnalazione` via `POST /segnalazioni`.
- 3 Il server salva su SQLite ed emette `segnalazione:nuova` a tutti i client.
- 4 Sul device del creatore la segnalazione viene aggiunta allo store in modo **idempotente** (niente doppioni con l'evento realtime).
- 5 Gli altri utenti ricevono l'evento: la Bacheca si aggiorna e parte una notifica locale.

8.3 Chat 1:1 proprietario ↔ ritrovatore

- 1 Un ritrovatore preme "Contatta" in Bacheca → naviga a `ChatScreen` con `{ idSegnalazione, idProprietario, idRitrovatore = me }`.
- 2 `ChatScreen` calcola la **chiave** `idSegnalazione:idRitrovatore`, carica i messaggi (`GET /messaggi?...&ritrovatore=...`) ed entra nella stanza `chat:<chiave>`.
- 3 All'invio, `ComunicazioneRealtimeUseCase` calcola il destinatario: se scrive il proprietario → destinatario è il ritrovatore; altrimenti → il proprietario.
- 4 Il server salva il messaggio ed emette `messaggio:nuovo` **solo** nella stanza della coppia: nessun altro utente della community lo vede.
- 5 Il proprietario vede tutte le sue conversazioni (una per ritrovatore) nella tab Messaggi.

Isolamento garantito. Poiché la stanza include il ritrovatore, due ritrovatori diversi che contattano lo stesso proprietario hanno conversazioni completamente separate.

9. Riferimento delle classi principali

Classe / modulo	Lato	Responsabilità
<code>AuthService</code>	client	Validazione credenziali, login/registrazione/logout, gestione JWT e sessione
<code>AttivaAllertaUseCase</code>	client	Logica di apertura allerta: crea bene + segnalazione, notifica i vicini
<code>ComunicazioneRealtimeUseCase</code>	client	Regole della chat: verifica segnalazione attiva, calcola destinatario, salva messaggio
<code>RegistraSmartIdUseCase</code>	client	Registrazione di un bene con controllo di unicità
<code>AllertaController</code> / <code>ChatController</code> / <code>SmartIdController</code>	client	Collegano UI e use case, gestiscono il dispatch su Redux
<code>Api*Repository</code>	client	Implementano le interfacce del dominio chiamando il server REST
<code>apiClient</code> / <code>socketClient</code>	client	Trasporto HTTP (con JWT) e canale WebSocket
<code>LocationService</code>	client	Posizione GPS dell'utente
<code>ExpoGatewayNotifiche</code> / <code>NotificationQueueService</code>	client	Invio notifiche push Expo
<code>authGuard</code> (jwt.ts)	server	Middleware di autorizzazione delle route
Router (<code>auth/users/smartid/segnalazioni/messaggi/conversazioni</code>)	server	Endpoint REST + emissione eventi realtime
<code>io.ts</code>	server	Socket.IO: stanze e funzioni di emit
<code>db.ts</code>	server	Connessione SQLite e schema

10. Glossario

Termine	Significato
JWT	Token firmato che identifica l'utente nelle richieste senza ri-loggarsi
bcrypt	Algoritmo di hashing per memorizzare le password in modo sicuro
WebSocket / Socket.IO	Canale bidirezionale persistente per il tempo reale
Room (stanza)	Gruppo logico di socket: l'emit a una stanza raggiunge solo i suoi membri
Repository	Oggetto che astrae l'accesso ai dati dietro un'interfaccia
Use case	Una specifica operazione applicativa (es. "attiva allerta")
Haversine	Formula per la distanza tra due coordinate geografiche
Idempotente	Operazione che, ripetuta, non produce effetti duplicati
Ritrovatore	Utente diverso dal proprietario che partecipa a una conversazione